Decision Orchestration: Step-by-Step Annotation of Complex Ontologies that Improves Itself from a Handful of Gold Sentences

Anonymous ACL submission

Abstract

Large language models (LLMs) can help bootstrap annotation when gold data is scarce using guidelines with few examples, but they often struggle with specialized annotation conventions and formats, while prompting alone is not reliable and interpretable on how to systematically revise instructions to correct their errors. We propose *decision orchestration*, a framework that turns annotation guidelines into an executable procedure in which code controls structure and models make local decisions. A reflection module serves as a moderation process, using decision traces and small batches of gold annotations to diagnose errors and iteratively refine the procedure.

On Chinese and English UMR and PARSEME, reflection with just a few annotated sentences improves 16 of 18 procedures using small language models or decision models. On the sampled English PARSEME test set, all reflected procedures outperform the best system from the 2018 shared task.

These findings support the framework's effectiveness and substantial potential for annotation under limited supervision, although gains vary across tasks and languages and do not consistently surpass strong baselines.

1 Introduction

Annotating a complex ontology takes two things: an understanding of the structure the scheme describes, and the scheme's own codebook, the conventions that make one annotation right and a plausible alternative wrong (Kim et al., 2026). Early in a project, a new scheme, language or domain has a guideline but little gold. With enough gold, supervised models learn both (Bevilacqua et al., 2021; Bai et al., 2022); with a handful of sentences they learn neither. Large language models bring the first ingredient for free: they understand sentences, events and referents well enough to bootstrap an annotation. What remains is to give them the codebook.

For simple labeling, the guideline fits into the prompt (Gilardi et al., 2023; Tan et al., 2024). Complex schemes pose two challenges: a) *guideline following*: the UMR guidelines exceed 300 pages (Van Gysel et al., 2021), burying relevant rules in long contexts (Liu et al., 2024) and an annotation must satisfy dozens of them at once, where following degrades with each added instruction (Harada et al., 2025). b)*interpretable moderation*: correcting annotation errors requires diagnosing and refining how guidelines are applied (Kim et al., 2026). When one prompt produces the whole annotation, errors are difficult to trace to specific decisions, leaving prompt revisions poorly targeted.

We propose **decision orchestration** (Figure 1), which compiles annotation guidelines into executable steps. Code enforces the output format, annotation inventory and structural constraints; models are consulted only at decision nodes, each with the part of the guideline it needs. Each node returns either a *fragment*, a schema-checked annotation from an LLM, or a *decision*, probabilities over candidate annotations from a decision model (TypeSafe, 2026; OpenAI, 2026; Microsoft, 2026). Each step is explicit and logged, allowing a reflector LLM to trace errors to the very steps that produced them and revise the responsible skills or the harness itself. This enables recursive self-improvement (RSI) of the procedure without changing model weights.

To our knowledge, this is the first work to annotate complex schemes step by step along their guidelines, and we show that it pays twice: (1) errors are diagnosable and can be fixed where they arise, and (2) with very little data, RSI makes the procedure much better than the guideline it started from and than a frontier model given the whole guideline.

We evaluate the framework with small LLMs

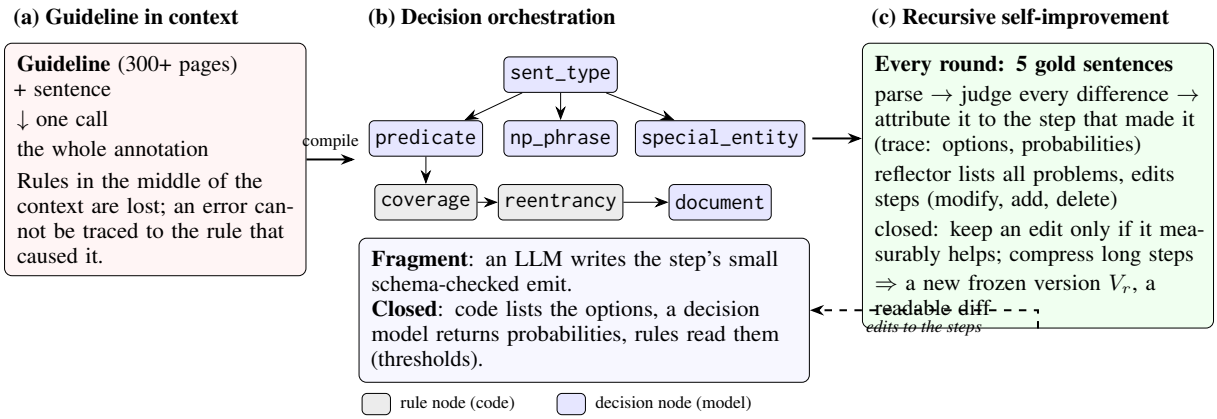


Figure 1: (a) The usual way to give an LLM a codebook: the whole guideline in context. (b) Decision orchestration compiles the guideline into a procedure of small steps, rule nodes and decision nodes, dispatched by a registry; a decision node is either a fragment decision by an LLM or closed questions answered by a decision model. (c) With a handful of gold sentences per round, a reflector edits the steps that made the errors.

and decision models on Chinese and English UMR parsing (Zeman and Štěpánek, 2026) and PARSEME verbal multiword expressions (Ramisch et al., 2018, 2020). Reflection with a few dozen annotated sentences improves most procedures across the two tasks and languages.

Our contributions are threefold:

- **An executable framework for annotation and moderation.** Decision orchestration combines code-enforced constraints with local model judgments. Explicit decision traces support moderation through targeted revisions to skills and the harness, enabling recursive self-improvement without changing model weights.
- **Empirical evidence under limited supervision.** Across Chinese and English UMR and PARSEME, we show that small language models and decision models can perform complex annotation through guideline-derived procedures. These procedures outperform a frontier prompting baseline in some settings, and reflection with 50 annotated sentences improves sixteen of eighteen completed procedures.
- **An analysis of how procedures improve and fail.** We trace errors to candidate generation, model judgments and structural rules, and examine the revisions made during reflection. The analysis reveals both concrete opportunities for improvement and limits that local model decisions alone cannot overcome.

Our contributions are:

- **An executable annotation framework.** Decision orchestration turns guidelines into inspectable procedures: code enforces annotation constraints, while small LLMs produce local fragments or decision models assign probabilities to candidate annotations.
- **Moderation through targeted revision.** Step-level records connect errors to the decisions that produced them, enabling a reflector to diagnose failures and revise the responsible skills or the harness itself. This supports recursive self-improvement of the procedure without changing model weights.
- **Evidence of both gains and limits.** Guideline-only procedures outperform a frontier model given the full guideline on Chinese UMR and PARSEME, and reflection improves sixteen of eighteen completed procedures by up to 31 F1 points. The gains are task-dependent: the frontier baseline remains stronger on English PARSEME, while closed procedures are limited by the candidates their code can generate. Decision traces expose these limits and identify concrete targets for improvement.

Our contributions are:

- **An executable annotation framework.** Decision orchestration combines code-enforced constraints with annotation fragments from small LLMs or probabilities from decision models. Without annotated examples, its Chinese UMR seeds using decision models outperform a frontier model given the full guideline by 9–11 F1 points.
- **Moderation through traceable decisions.** Step-level records enable a reflector to diagnose errors

and revise skills or the harness itself. Our analysis shows how candidate-generation failures limit closed procedures and how measuring proposed edits helps control reflection drift.

- **Improvement with minimal supervision.** Reflection using only 50 annotated sentences improves sixteen of eighteen completed procedures, by up to 31 F1 points. On the sampled English PARSEME test set, all reflected procedures outperform the best system from the 2018 shared task, although the frontier prompting baseline remains stronger.

Our findings:

- **Steps beat the guideline in context.** Without any gold, the best seed procedure matches or beats GPT-6.1-sol given the whole guideline on three of four task–language pairs; on Chinese UMR every closed seed is 9–11 points ahead on the official ju:mætʃ F1 (Jev 33.9 against 23.9), although a content-based metric, AnCast++, still puts the frontier model ahead (§5). On English PARSEME, a scheme close to what frontier models already know, the frontier model stays ahead (§5).

- **A handful of gold goes a long way.** Ten rounds of reflection on five gold sentences each improve sixteen of eighteen procedures, by up to 31 points (Jev on English PARSEME, 22.5 → 53.7). The best UMR procedure (Chinese, 37.6) is decided by Jev at under 30 cents per hundred sentences, and on English PARSEME every reflected procedure is above the best shared-task system rescored on the same sentences.

- **Errors have addresses.** The step-level judge shows where the two kinds of decision fail: closed procedures lose where the code offers no right option (e.g., an English lemmatizer that leaves *stolen* unreduced), fragment procedures where structure is left to the model; measured edits keep closed reflection from drifting, unmeasured fragment edits do not (§5.1, §6).

These results suggest that LLMs can learn an annotation scheme from its instructions and a few dozen gold sentences, keep improving as new gold arrives, and that the models doing the annotating need not be frontier models: even simple decision models can serve.

2 Related Work

LLMs as annotators LLMs can replace or assist crowd workers on many labeling tasks (Wang et al., 2021; Gilardi et al., 2023; Törnberg, 2023; He et al., 2024; Ziems et al., 2024; Tan et al., 2024), but their quality varies by task and must be validated against human labels (Pangakis et al., 2023). Zero-shot LLM labels often miss a benchmark's specific conventions. Prior work narrows this gap by adding guidelines to prompts and refining them through moderation (Kim et al., 2026), learning guidelines from errors (Pang et al., 2023), or training models to follow guidelines written as code (Sainz et al., 2024; Wang et al., 2023). Yet it remains unclear how a model can apply a long guideline reliably as all of these keep the guideline as one block of context for one model call.

For complex ontologies such as UMR, whose guidelines run to hundreds of pages (Van Gysel et al., 2021; Bonn et al., 2025), that block exceeds what a model reliably uses: performance falls for information in the middle of long contexts (Liu et al., 2024) and drops roughly exponentially with the number of instructions to be followed at once (Harada et al., 2025). Expert-level structured annotation by prompting remains weak (Ettinger et al., 2023; Sun et al., 2024), and supervised parsers need large corpora (Chun and Xue, 2024; Markle et al., 2026). Together, these limitations leave a gap: how to apply long annotation guidelines reliably without a large labeled corpus.

Recursive self-improvement and rule-based reflection Early NLP systems improved annotation by learning rules from gold labels (Brill, 1992, 1995; Ramshaw and Marcus, 1995), but relied on predefined rule templates. LLMs have renewed this idea with more flexible revisions of prompts (Pryzant et al., 2023; Khattab et al., 2024), rule sets (Zhang et al., 2026b; Zhao et al., 2024), or harness codes(Zelikman et al., 2024; Hu et al., 2025; Zhang et al., 2026a). However, their use in expert structured annotation remains underexplored, especially when annotations must follow long, complex guidelines. The limits of self-correction without external feedback (Huang et al., 2024) motivate a structured harness that constrains revisions and checks whether they improve annotation accuracy.

3 Methodology: Decision Orchestration

Workflow and components A registry runs rule nodes and decision nodes to produce an annotation and an execution trace. On gold data, a judge scores individual annotation units, and a reflector uses the scores and trace to revise the procedure. We represent this system as

$$\begin{aligned} \Pi &= (\mathcal{S}, R, C, M, W, \Theta, J, \mathcal{F}), \\ s_i &= (I_i, O_i, d_i, T_i) \in \mathcal{S}, \\ \mathcal{F} &= (F_{\text{diag}}, F_{\text{valid}}, F_{\text{comp}}). \end{aligned} \tag{1}$$

Here, $\mathcal{S}$ contains decision nodes s_i, each implemented as a skill with input and output schemas I_i, O_i, instructions d_i, and available tools T_i. R is the registry, C the deterministic code, W the worktree, M the annotation and reflection models, and Θ numerical parameters such as thresholds. J is the set of fine-grained judges; $\mathcal{F}$ comprises diagnosis, edit validation and compression.

R executes nodes in ordered groups (*waves*), dispatching a rule node in C or a decision node in $\mathcal{S}$(*emits*), categorizes outputs to worktree W or to next nodes, and assembles the final outputs from the worktree W. At s_i, M evaluates the input under d_i, with supporting information from T_i; C validates or converts its answer to an emit conforming to O_i, using Θ for closed decisions, and assembles the annotation. J scores it against gold in fine-grained aspects, along with a *comprehensive score* defined by the annotation benchmark; $\mathcal{F}$ invokes a reflector in M to locate errors, propose edits to the responsible instructions or code, then validates and compresses the revised procedure to avoid skill instructions from becoming too verbose.

Modes of Decision nodes Decision nodes have two implementations with the same emit. In **fragment mode**, an LLM reads the skill's instructions and input, uses its available tools, and returns a graph fragment in structured outputs. In **closed mode**, the system uses codes to enumerate options and asks choice, yes/no or graded questions. A decision model returns probabilities over these options, which rules convert into the emit using thresholds in Θ. Both modes log inputs and outputs; closed mode also records options and probabilities.

Cascading For closed decisions with confidence lower than a *cascading threshold*, a **cascading** mechanism routes the input to a stronger model that receives the same inputs and replaces the original answer.

Calibration For the *cascading threshold* above, and to account for different decision models' output probabilities for*decision thresholds*, we fit both in 0.1 increments on a small batch of training annoations for a best comprehensive score.

Fine-grained evaluation The judges in J score individual annotation units against gold; execution traces link predictions to the responsible steps, providing targeted feedback for reflection. Task-specific checks and scoring details are given in the Metrics paragraph of Experiments (§4).

Reflection Reflection revises the procedure in three steps: **Diagnose (F_{diag}):** use judge feedback and execution traces to locate errors and propose targeted edits. **Validate (F_{valid}):** check candidate revisions and retain those that satisfy the acceptance criteria. **Compress (F_{comp}):** merge redundant instructions and rules while preserving required options and conventions. The experimental protocol is detailed in §4 (Reflection protocol).

4 Experiments

Tasks We evaluate two structured annotation tasks (Figure 2). **UMR** maps sentences to token-aligned graphs of concepts, roles and attributes, and adds document-level temporal, modal and coreference relations (Van Gysel et al., 2021; Bonn et al., 2025). **PARSEME** identifies verbal multiword expressions (VMWEs), including discontinuous expressions, and assigns each a category (Ramisch et al., 2018, 2020).

Data For UMR, we use Chinese and English data from the DMR 2026 shared task, largely based on UMR 2.1 (Zeman and Štěpánek, 2026). We select 50 training sentences per language in ten batches of five; the test sets contain 100 Chinese and 95 English sentences. We use shared-task tokenization. For PARSEME, we use Chinese edition 1.2 (Ramisch et al., 2020) and English edition 1.1 (Ramisch et al., 2018). Each language has 50 training sentences in ten batches of five (three with a VMWE and two without per batch) and 100 sampled official test sentences (50 with a VMWE and 50 without). Table 1 lists the decision skills and rule nodes, Figure 3 shows a UMR trace, and Appendix A gives the seed instructions.

(a) UMR: a sentence graph, its alignment, and document-level relations

And$_1$ *you*$_2$ *watch*$_3$ *him*$_4$ *pluck*$_5$ *a*$_6$ *few*$_7$ *pears*$_8$ *,*$_9$ (sentence 10 of a narrative)

```
(s10w / watch-01
 :ARG0 (s10p / person
    :refer-person 2nd :refer-number singular)
 :ARG1 (s10p2 / pluck-01
    :ARG0 (s10p3 / person
       :refer-person 3rd :refer-number singular)
    :ARG1 (s10p4 / pear :quant (s10a / a-few))
    :aspect activity)
 :aspect state)
```

```
alignment: s10w 3-3, s10p 2-2, s10p2 5-5,
 s10p3 4-4, s10p4 8-8, s10a 6-7
:temporal ((s8e :after s10w)
       (s10w :overlap s10p2))
:modal ((root :modal author)
     (author :full-affirmative s10w) ...)
:coref ((s8p :same-entity s10p)
     (s9p2 :same-entity s10p4))
```

(b) PARSEME: verbal multiword expressions

After that, he made$_6$ *a great effort*$_9$ *to be calm.*
$\{6, 9\}$: `LVC.full` (light verb + predicative noun, discontinuous)

So Harry had been brought$_5$ *up*$_6$ *by his dead mother's sister ...*
$\{5, 6\}$: `VPC.full` (the particle changes the verb's meaning)

我 已經 作出$_3$ 決定$_4$ ， 讓 他 發言 。
$\{3, 4\}$: `LVC.full`; $\{3\}$: `VPC.semi` (one token: 作 + complement 出)

Categories (EN 1.1): VID, LVC.full, LVC.cause, VPC.full, VPC.semi, IAV;
(ZH 1.2): VID, LVC.full, LVC.cause, VPC.semi, MVC.

Figure 2: Training examples of (a) UMR sentence graphs, token alignments and document-level relations, and (b) PARSEME verbal multiword expressions, which may be discontinuous or overlap; in Chinese, one token can form an expression.

Scheme	Node	Kind	Responsibility
UMR	`sent_type`	Decision	Classify a span and route it; emit a graph for a leaf.
	`discourse`	Decision	Split a span into clauses and assign their discourse relation.
	`predicate`	Decision	Choose the head, sense, aspect, modal strength, polarity and argument roles.
	`np_phrase`	Decision	Annotate noun-phrase heads, modifiers, roles, quantity and reference attributes.
	`special_entity`	Decision	Annotate names, dates, quantities and ordinals.
	`document`	Decision	Assign temporal, modal and coreference relations.
	`coverage`	Rule	Return uncovered content tokens to sentence routing.
	`antecedent`	Rule	Record participants outside the current span for later merging.
	`reentrancy`	Rule	Merge nodes with matching types and spans and resolve outside participants.
PARSEME	`candidates`	Rule	Propose token groups from forms, lemmas, parts of speech and dependencies.
	`decide`	Decision	Identify and categorize expressions: return token groups in fragment mode, or answer linguistic tests in closed mode.

Table 1: Decision skills use instructions and model M; rule nodes are implemented in code C. Both schemes use code to assemble the final output.

Baseline We use GPT-6.1-sol to generate the annotation in a single call per test chunk, given the full guideline, the input sentences (including lemma, part-of-speech and dependency columns for PARSEME) and the required output format. The guideline itself contains examples; we provide no separate gold examples.

Seeding We write the seed procedures solely from the published guidelines before reading any training gold. Neither mode includes worked examples (Appendix A). In fragment mode, decision skills restate the relevant guideline in natural language and return schema-constrained graph fragments or expressions. In closed mode, we encode possible outputs as fixed-choice questions, with guideline tests, thresholds and deterministic branches; the model supplies choices and code assembles the annotation.

Models and batches Each training batch contains five sentences. Fragment-mode parsing uses GPT-6-luna and Claude Haiku 5.5; Claude Sonnet 5.5 is also tried as a fragment decider, for two rounds on UMR and three on PARSEME. Closed-mode parsing uses Jev 1.13 (TypeSafe, 2026), OpenAI Decisions with `gpt-6-luna-decisions` (OpenAI, 2026), and Microsoft-Decision-1 (Microsoft, 2026). Claude Sonnet 5.5 performs reflection. OpenAI Decisions is called through OpenAI's API, Sonnet's UMR decisions through the Claude subscription, and all other models through OpenRouter.

Reflection protocol The reflector reads gold and predicted annotations, and unit-level scores, attributed errors, decision traces and editable files, then proposes additions, replacements or deletions to instructions (Algorithm 1). In closed mode, each edit is applied to a copy and evaluated on the current and all earlier training batches. It is kept if current-batch comprehensive score improves and earlier-batch score falls by at most 0.5, or earlier-batch score improves without a current-batch loss. Rejected edits are returned once with their mea-

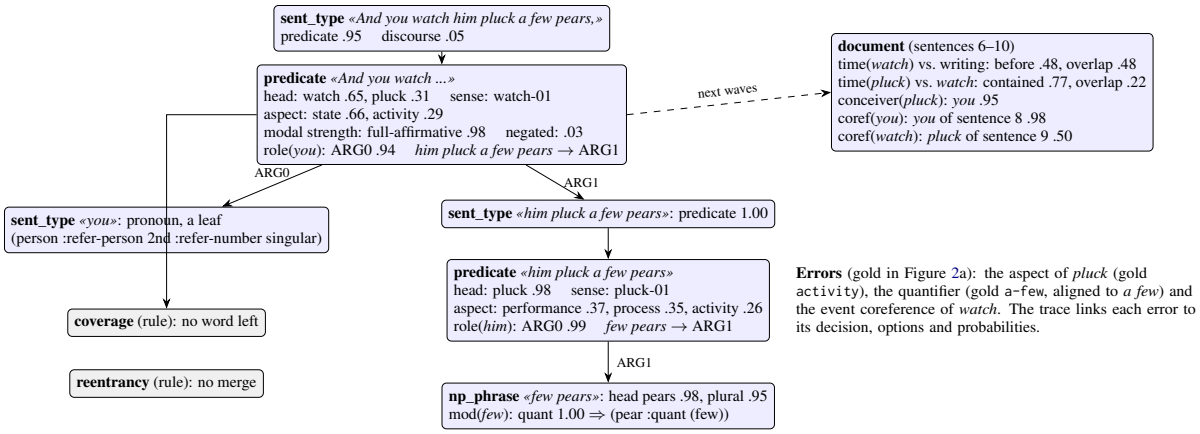


Figure 3: A real trace of the closed-mode UMR seed with Jev. Blue: decision nodes with options and probabilities; grey: rule nodes. The callout identifies decisions that differ from gold (Figure 2a).

sured effects. In fragment mode, edits are accepted if they apply and leave the procedure loadable.

Files exceeding 1.5× their seed length are compressed, merging redundant rules while preserving output format parts like options, questions and conventions.

Metrics UMR is scored with the shared task's official ju:mætʃ F1 (Zeman and Štěpánek, 2026), where predicted nodes are mapped to gold nodes through their token alignment, and F1 is computed over the triples (concepts, relations, attributes) of mapped nodes, document-level relations included. For diagnosis we use a fine-grained judge (Table 2): it maps nodes with AnCast (Sun and Xue, 2024), breaks the comparison into 25 checks in 11 modules, gives every unit a verdict and every check its own F1, and attributes each predicted unit to the skill call that produced it. PARSEME is scored with the official evaluation suite *MWE-based F1* (exact token set and category), token-based F1, and F1 on expressions unseen in the training data. A PARSEME judge sorts every gold and predicted expression into six kinds: correct, wrong category (same tokens), partial (overlapping tokens), missed without a candidate (no rule proposed its tokens), missed after rejection (a candidate had them), and spurious (no overlap with any gold expression).

SOTA For UMR, we report the best shared-task submission (Zeman and Štěpánek, 2026), Orange/08 (36.5 ju:mætʃ in Chinese, 22.2 in English), on the full blind test set. We have the test gold but not its predictions, so we cannot re-score it on our chunks. Chun and Xue (2024) (Chun and Xue, 2024) and SETUP (Markle et al., 2026) evaluate on UMR 1.0 and 2.0, respectively, and use externally trained components or additional training data; we therefore exclude their scores from this comparison. For PARSEME, we re-score the shared-task systems on our sentences (Chinese 1.2: TRAVIS-mono/-multi; English 1.1: Milos and TRAPACC). On the full test files our scoring reproduces their published F (71.9 for TRAVIS-mono, 33.3 for Milos); on our sample, which has as many sentences with a VMWE as without, the same systems score higher (TRAVIS-multi 81.1, TRAVIS-mono 80.7, Milos 40.5). The best published English 1.1 result (Rohanian et al., 2019) is quoted on the full test set.

5 Results

Table 3 gives the headline numbers, Figure 4 every version along the way, and Appendix Table 7 every number.

Steps beat the guideline in context. Without any gold, the closed seeds are 9–11 points above GPT-6.1-sol on Chinese UMR (33.2–35.0 against 23.9), and within three points of it on English UMR. On Chinese PARSEME the fragment seeds start well above it (59.5–72.1 against 50.0); on English PARSEME, a scheme close to what frontier models know, every seed stays below it (75.6).

Fifty gold sentences of reflection. Reflection improves sixteen of the eighteen ten-round chains. The largest gains are on PARSEME with measured closed edits: Jev 37.8 → 63.4 (Chinese) and 22.5 → 53.7 (English), OpenAI Decisions 35.1 → 54.5 and 42.0 → 56.6. On UMR, Jev reaches 37.6 in Chinese, the best UMR procedure, and Haiku 32.6 in English; Haiku's Chinese seed gains 6.9 points

Module	Checks	What one unit compares (for matched gold/predicted nodes)	Unit kind
a. nodes	event found, entity found	does a gold event (entity) node have a predicted match; is a predicted one matched	node
	event concept, entity concept	is the concept the same	node pair
	granularity	is the node aligned to the same words (one node for several gold nodes or vice versa)	span
b. sense	event sense	same PropBank roleset number, once the concept is right	node pair
c. attributes	aspect, modal strength, polarity, refer, other	same value of the attribute; an attribute only one side has is missing or extra	attribute
d. arguments	core args of events / of abstract predicates; peripheral args of events / of entities	how far apart the argument sets of a matched head are (credit = share that agrees)	head
e. role labels	core, peripheral	same role label on a matched edge	edge
f. names	name tokens, wiki	same :op tokens of a matched name; same wiki title	entity
g. reentrancy	reentrancy	is an edge back to an already declared node also in gold	edge
h. temporal	temporal dependency	a child node's temporal parent(s) and relation(s)	child
i. modal	modal dependency	an event's conceiver (parent) and modal strength (label)	child
j. coreference	chains, subsets	is a mention linked to the same other mentions as in gold; subset relations	mention
k. format	document format	are the predicted document-level relations well formed (no gold needed)	relation

Table 2: The fine-grained UMR judge. Every unit gets a verdict: correct (credit 1), partial (0.5, or the check's own credit), wrong, missing or extra (0). Recall counts the gold side (correct, partial, wrong, missing), precision the predicted side (correct, partial, wrong, extra), and each check reports its F1. Modules a–f never see reentrant edges, so module g's errors do not move them. Each predicted unit is attributed to the skill call that wrote it.

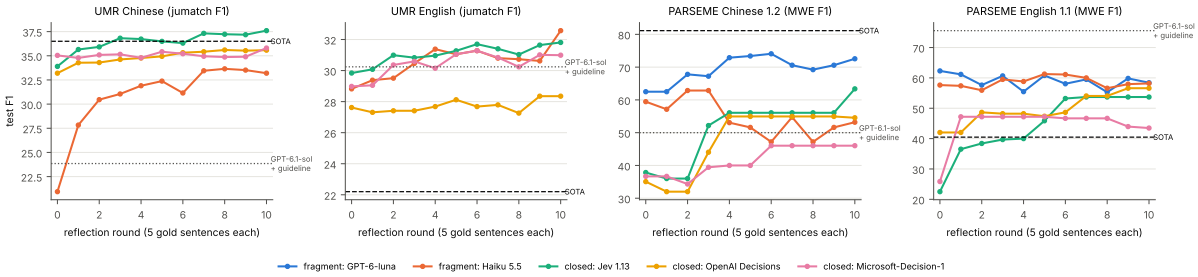


Figure 4: Test F1 of every frozen version $V_0, \ldots, V_{10}$; round r has read $5r$ gold sentences. Dotted: GPT-6.1-sol with the whole guideline; dashed: best shared-task system (UMR: as published, on whole documents).

in round 1 alone, by learning that Chinese concepts keep the source word. Unmeasured fragment edits are less stable: Haiku's Chinese PARSEME procedure peaks at 62.9 (V_3) and ends at 53.2. A larger fragment decider, Sonnet 5.5, gives the best Chinese PARSEME procedure (75.9 after three rounds) and the best English UMR scores on its 25-sentence test.

Against the state of the art. On English PARSEME every reflected procedure is above the best 1.1 system on the same sentences (Milos, 40.5), though not above the frontier model in context. On Chinese PARSEME the 1.2 systems, trained on the full data, stay ahead (81.1 against 75.9). On UMR our best procedures (37.6 Chinese, 32.6 English) score above the best shared-task submission's published results (36.5, 22.2), on smaller evaluation units (five-sentence chunks against whole documents), so the comparison is indicative. The comparison with the frontier model also depends on the metric: AnCast++, which matches nodes by content instead of token alignment, puts GPT-6.1-sol first (44.8 Chinese, 45.8 English, against 42.2 and 37.5 for Haiku's V_{10}). Our procedures get their alignments from code, so their advantage on UMR holds on the official metric, not on graph content alone.

5.1 Where fragment and closed decisions differ

Because every unit is attributed to the step that made it, reflection's gains and the two modes can be compared check by check (Figure 5). Reflection raises 29 of the 32 Chinese scores and 28 of the 32 English ones, most of all where the seed was weakest (Haiku's Chinese event concepts 30.0 → 77.3, entity concepts 36.3 → 73.8, core arguments 22.2 → 45.6). At V_{10}, closed decisions win where code controls the output (node granularity, Chinese 92.9 against 85.8; person and number, 49.0 against 23.2), fragment decisions where judgment is open (event arguments, 45.6 against 35.7; coreference, 16.4 against 12.7). In English, Jev loses on event concepts (55.6 against 72.5), and the trace shows why: 51 of its 212 wrong concepts are inflected forms (*stolen*, *thought*) that the seed's rule-based lemmatizer did not reduce. A decision model can only pick among the options code offers, so the closed mode is as good as its enumerator; such an error is found in one column of the judge and is

System	UMR (juːmætʃ F1)		PARSEME (MWE F1)	
	Chinese	English	Chinese	English
Best shared-task system	36.5†	22.2†	81.1	40.5
GPT-6.1-sol + full guideline	23.9	30.2	50.0	**75.6**
fragment: GPT-6-luna, seed V_0	18.9‡	24.4‡	62.5	62.3
V_{10}	28.4 (V_2)‡	34.3 (V_2)‡	72.5	58.4
fragment: Haiku 5.5, seed V_0	20.9	28.8	59.5	57.6
V_{10}	33.2	**32.6**	53.2	58.2
fragment: Sonnet 5.5, seed V_0	29.7‡	39.7‡	72.1	58.5
V_{10}	35.0 (V_2)‡	42.5 (V_2)‡	**75.9** (V_3)	56.5 (V_3)
closed: Jev 1.13, seed V_0	33.9	29.8	37.8	22.5
V_{10}	**37.6**	31.8	63.4	53.7
closed: OpenAI Decisions, seed V_0	33.2	27.6	35.1	42.0
V_{10}	35.6	28.4	54.5	56.6
closed: Microsoft-Decision-1, seed V_0	35.0	29.0	36.7	25.9
V_{10}	35.8	31.0	46.0	43.5

Table 3: Test F1. UMR: the shared task's juːmætʃ metric on 100 Chinese and 95 English sentences; PARSEME: MWE-based F1 on 100 sentences per language. V_0 is the seed written from the guideline, V_{10} the procedure after ten rounds of reflection on five gold sentences each (50 sentences in all); a version in parentheses is the last round a shorter run reached. Bold: the best result in each column among the systems we ran. †Best submission of the UMR shared task (Orange/08) as published for the full blind test set, scored on whole documents, not re-scored on our five-sentence chunks. PARSEME: the best shared-task system on our 100 test sentences, re-scored with the official script from the system's released predictions (trained on the full training data): TRAVIS-multi for Chinese 1.2 (70.0 F on the full test set, where TRAVIS-mono's 71.9 is the best published result) and Milos for English 1.1 (33.3 on the full test set). Our sample has as many sentences with a VMWE as without, so almost every system scores higher on it than on the full test set, where most sentences have none. ‡Tested on 25 of the test sentences (five chunks) after two rounds of reflection, as the full runs did not fit our time and rate limits; not comparable with the other UMR cells.

cheap to fix (an irregular-verb table).

On PARSEME (Figure 6), the closed seeds start low because they accept far too many candidates (78–123 spurious predictions for 57–59 gold expressions). Reflection teaches them the scheme's sense of "light" and "idiomatic" ("有 is not a light verb"), spurious predictions fall in all six closed chains (Jev: 80 → 9 in Chinese, 112 → 19 in English; Microsoft, English: 123 → 11), and correct ones rise in five of the six. What closed procedures cannot recover are expressions no rule proposes (11–16 missed, against 3–7 for fragment procedures).

5.2 Calibration and transfer

Threshold calibration (§3) helps six of the eight seeds but leaves the reflected procedures unchanged or worse (Table 4; within ±0.2 on UMR, Jev Chinese PARSEME 63.4 → 61.4): measured reflection has already tuned the thresholds, and refitting them on 50 sentences overfits. The procedure Jev's chain learned also transfers: answered by Microsoft-Decision-1 it beats Microsoft's own V_{10} in three of four cases (Chinese PARSEME 57.1 against 46.0; 62.0 once calibrated), so what reflection writes is mostly about the scheme, not the model. A third adjustment, cascading uncertain decisions to a small LLM, is in Appendix C.

5.3 What reflection does

The reflector lists 9–11 problems per UMR batch and 2–4 per PARSEME batch and proposes about one edit per problem (Table 5). Measurement keeps about half of the closed-mode edits; fragment mode keeps every edit that applies, so its procedures grow faster (Chinese UMR 36.8k → 50.8k characters, against 70.5k → 73.7k in closed mode). The kept edits read like a manual's errata ("`exist-91` for X中有Y", "speech content is not discourse", "add inceptive aspect"), each a readable change to a named step that an annotator can check, keep or revert.

6 Discussion

Diagnosability. A guideline in context fails silently: nothing in GPT-6.1-sol's output says which part of the 366k-character guideline it misread. Each error of a procedure has an address (a step, its input, its options and their probabilities), so it can be found and fixed where it arises. The Haiku seed's English concepts for Chinese words, Jev's unlemmatized English verbs and the closed PARSEME seeds' spurious light-verb constructions (§5.1) were each found in one column of the judge's output and traced to one file. The same addresses let the reflector read the decisions behind each error, and its edits are diffs that an annotator can review, keep or revert.

Decision models are cheap and promising. Decision models return probabilities over options that code has listed, which makes them the cheapest deciders we tried: about $0.1–0.3 per hundred UMR sentences, against $1.9 for Haiku and $7.1 for GPT-6.1-sol with the guideline (Chinese). Closed parses are fast enough to measure every edit, which keeps closed-mode reflection from drifting, and Jev gives the best UMR procedure in Chinese. Their ceiling is set by the code around them: a decision model cannot pick an option that is not offered. Better

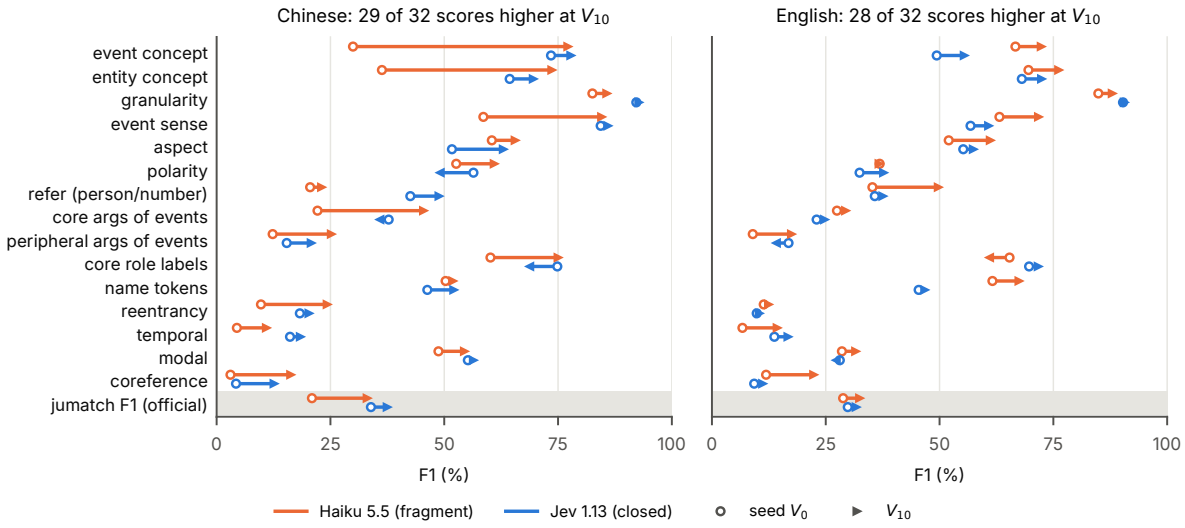


Figure 5: The fine-grained UMR judge: F1 of each check for fragment decisions by Haiku and closed decisions by Jev, from the seed V_0 (circle) to V_{10} (arrowhead); shaded: the official juːmætʃ F1. Selected checks; numbers in Appendix Table 8, all 25 checks in the released results.

	UMR zh		UMR en		PARSEME zh		PARSEME en	
Procedure	raw	cal.	raw	cal.	raw	cal.	raw	cal.
Jev 1.13 V_0	33.9	34.2	29.8	30.2	37.8	39.2	22.5	27.1
Jev 1.13 V_{10}	37.6	37.4	31.8	31.8	63.4	61.4	53.7	49.5
Microsoft-Decision-1 V_0	35.0	35.8	29.0	29.0	36.7	35.0	25.9	29.9
Microsoft-Decision-1 V_{10}	35.8	36.0	31.0	31.1	46.0	46.0	43.5	43.5
Jev V_{10} answered by Microsoft	36.6	–	30.4	–	57.1	62.0	44.2	42.3

Table 4: Closed procedures with their thresholds as written (raw) and calibrated on the 50 training sentences (cal.); Jev's V_{10} answered by Microsoft-Decision-1 (on UMR not calibrated, for budget).

Task	Decisions	problems	edits	kept	compr.	size V_0	size V_{10}
UMR zh	Haiku 5.5	108	57	55	28	36.8k	50.8k
	Jev 1.13	108	109	54	0	70.5k	73.7k
	OpenAI Decisions	110	102	31	0	70.5k	73.3k
	Microsoft-Decision-1	114	106	54	0	70.5k	74.8k
UMR en	Haiku 5.5	100	59	58	17	36.8k	51.5k
	Jev 1.13	87	89	35	0	70.5k	72.6k
	OpenAI Decisions	78	81	27	0	70.5k	71.5k
	Microsoft-Decision-1	85	71	35	0	70.5k	72.7k
PARSEME zh	GPT-6-luna	26	20	18	12	4.5k	8.0k
	Haiku 5.5	29	18	18	9	4.5k	7.5k
	Jev 1.13	37	38	16	2	8.0k	9.6k
	OpenAI Decisions	37	59	13	0	8.0k	9.3k
	Microsoft-Decision-1	38	46	13	0	8.0k	9.2k
PARSEME en	GPT-6-luna	24	18	18	8	4.7k	8.1k
	Haiku 5.5	26	21	21	9	4.7k	8.1k
	Jev 1.13	33	36	13	0	8.1k	9.7k
	OpenAI Decisions	40	50	14	2	8.1k	10.0k
	Microsoft-Decision-1	38	31	8	0	8.1k	9.2k

Table 5: Ten rounds of reflection: problems the reflector listed, edits tried (with closed-mode retries), edits kept, compressions, and the size of the editable procedure (characters; closed mode includes code).

enumerators raise that ceiling for every decision model at once, and the transfer result (§5.2) suggests that a learned procedure can move between decision models.

What reflection can and cannot do. With five gold sentences a round, reflection writes the conventions a guideline leaves implicit (Chinese concepts keep the source word; 有 is not a light verb) and fixes procedural errors, with the largest gains in the first rounds. Three limits are visible. (1) Unmeasured fragment edits drift (Haiku's Chinese PARSEME procedure loses almost ten points after round 3); measuring them would need cheap fragment decisions. (2) The training batches are tiny, so the gap between training and test grows and threshold refitting overfits; more gold strengthens both checks. (3) Reflection edits what exists and rarely changes the decomposition itself, which is where English PARSEME, close to what frontier models already know, stays below the guideline in context.

When to decompose. The frontier model in context is strongest where the scheme is close to what such models know and the output is short (English

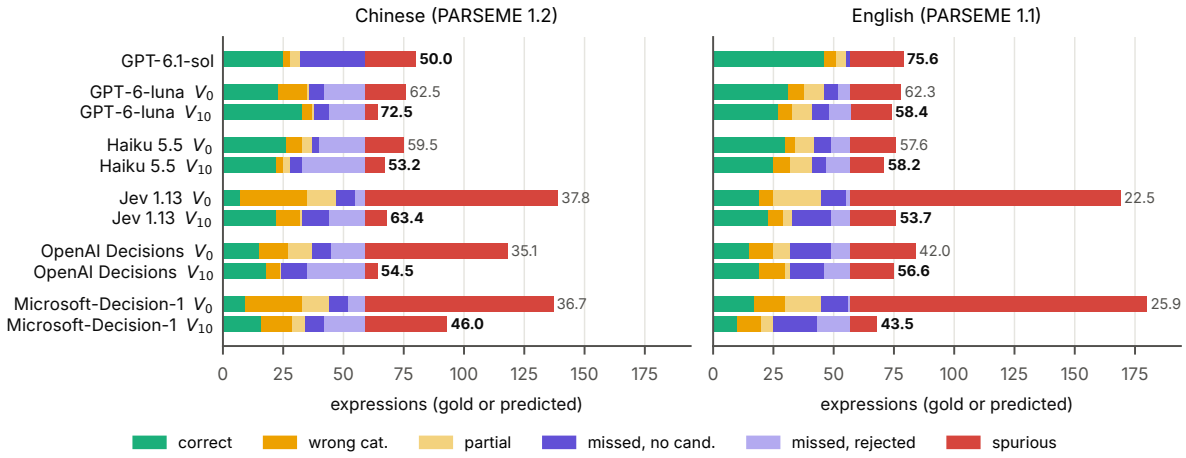


Figure 6: PARSEME error kinds on the test set, seed V_0 and V_{10} of each procedure (counts of gold or predicted expressions): correct; same tokens with the wrong category; overlapping tokens; gold missed because no rule proposed its tokens, or because a proposed candidate was rejected; spurious predictions. At the end of each bar: MWE-based F (category-blind, as in the official script), bold for V_{10} and the baseline. Numbers in Appendix Table 9.

verbal MWEs); decomposition is strongest where the output is structure and the conventions are particular (Chinese UMR). For a new scheme or language early in a project, a procedure costs a few dollars to run, improves from the first batches of gold, and leaves a trace in which every error can be found.

7 Conclusion

We annotated two complex schemes step by step along their guidelines. Compiled into a procedure of rule nodes and small decisions, a guideline that is too long to follow in context becomes something small and cheap models can follow: seeds written from the guideline alone, with no example, match or beat a frontier model reading the whole guideline on three of four task–language pairs, 9–11 points ahead of it on Chinese UMR on the official metric (on graph content alone, measured by AnCast++, the frontier model stays ahead). Every error has an address, so it can be traced to the step that made it, and a reflector can fix the step: from fifty gold sentences, five at a time, the procedures improved in sixteen of eighteen runs, by up to 31 points, and on English PARSEME every reflected procedure is above the best shared-task system on the same sentences; on UMR the best procedures also score above the best shared-task submission's published results (37.6 against 36.5 in Chinese, 32.6 against 22.2 in English), though on smaller evaluation units. Decision models that only return probabilities, at a few cents per hundred sentences, take the best UMR result. The procedure keeps a readable history of what it learned, and the more gold arrives, the more it has to learn from. We think this makes decision orchestration a natural first step for a new annotation project: start from the guideline, annotate with cheap models, and let the procedure improve as the first gold comes in.

Limitations

Scale. Test sets are 95–100 sentences per task and language and training is 50 sentences; each chain was run once, with one document order. With 57–59 gold expressions per PARSEME test set, one expression is worth about one point of F, so differences of a few points there should not be overread; we did not repeat chains with other document orders to measure the variance of reflection. **Unfinished runs.** OpenRouter limits each OpenAI model to 20 requests a minute for our account, and our API budget was fixed; GPT-6-luna as a UMR fragment decider needs about 3,000 calls per test pass, so we ran it on UMR for two rounds only, tested on 25 sentences, as we did for Sonnet 5.5 as a fragment decider (marked in Table 3). OpenAI Decisions was run through OpenAI's own API, the other models through OpenRouter. **Data editions.** PARSEME 2.0 has no Chinese data and only an English trial set, so we use editions 1.2 (Chinese) and 1.1 (English); the English 1.1 systems predate pretrained models. There is no UMR 3.0 release;

we use the 2026 shared-task data, which is based on UMR 2.1, and the shared task's best results are quoted as published, on whole documents, not rescored on our five-sentence chunks. **Seeds.** The seeds were written by the authors (with an LLM's help) from the guidelines; their quality affects every result and we have not measured the effort they take. Closed and fragment seeds share the decomposition, so the comparison between modes is not a comparison of decompositions. **Reflection.** Fragment-mode edits are accepted unmeasured and drift; the same reflector (Claude Sonnet 5.5) was used throughout. **Models.** All models are commercial services whose behavior may change; we release the predictions, the procedures at every version, and the cached answers that their terms permit.

Ethics Statement

We use the UMR shared-task data and PARSEME 1.1 and 1.2 under their licenses (PARSEME 1.2 Chinese: CC BY-NC-SA 4.0) and collect no data from people. The method aims to make the first annotations of a new scheme cheaper and easier to check; it could reduce demand for annotation work, and we see it rather as a way to direct annotators to the decisions that need them. Automatic annotations should be checked before they are used as gold. The runs of this version cost about $59 in logged API charges (decision services $13, LLM calls $46), including runs that were stopped; reflection used a subscription. Generative AI assisted the implementation, the running of the experiments and the editing of this paper; the authors are responsible for all content.

References

Xuefeng Bai, Yulong Chen, and Yue Zhang. 2022. Graph pre-training for AMR parsing and generation. *arXiv preprint arXiv:2203.07836*.

Michele Bevilacqua, Rexhina Blloshmi, and Roberto Navigli. 2021. One SPRING to rule them both: Symmetric AMR semantic parsing and generation without a complex pipeline. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(14):12564–12573.

Julia Bonn, Claire Bonial, Matt Buchholz, Hsiao-Jung Cheng, Alvin Chen, Ching-wen Chen, Andrew Cowell, William Croft, Lukas Denk, Ahmed Elsayed, Eva Fučíková, Federica Gamba, Carlos Gomez, Jan Hajič, Eva Hajičová, Jiří Havelka, Loden Havenmeier, Ath Kilgore, Veronika Kolářová, and 40 others. 2025. Uniform meaning representation 2.0.

Eric Brill. 1992. A simple rule-based part of speech tagger. In *Proceedings of the Third Conference on Applied Natural Language Processing*, pages 152–155. Association for Computational Linguistics.

Eric Brill. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics*, 21(4):543–565.

Jayeol Chun and Nianwen Xue. 2024. A pipeline approach for parsing documents into uniform meaning representation graphs. In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*.

Allyson Ettinger, Jena D. Hwang, Valentina Pyatkin, Chandra Bhagavatula, and Yejin Choi. 2023. "you are an expert linguistic annotator": Limits of llms as analyzers of abstract meaning representation. *arXiv preprint arXiv:2310.17793*.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Keno Harada and 1 others. 2025. When instructions multiply: Measuring and estimating LLM capabilities of multiple instructions following. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics.

Xingwei He, Zhenghao Lin, Yeyun Gong, A-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 165–190, Mexico City, Mexico. Association for Computational Linguistics.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, and 1 others. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*.

Kon Woo Kim, Jin-Dong Kim, and Akiko Aizawa. 2026. Refining and reusing annotation guidelines for LLM

annotation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 37951–37964, San Diego, California, United States. Association for Computational Linguistics.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Emma Markle, Javier Gutierrez Bach, and Shira Wein. 2026. SETUP: Sentence-level english-to-uniform meaning representation parser. In *Proceedings of the Fifteenth Language Resources and Evaluation Conference*, pages 9710–9721.

Microsoft. 2026. Use Microsoft-Decision-1 in Azure AI Foundry. https://learn.microsoft.com/en-us/azure/foundry/foundry-models/how-to/use-foundry-models-microsoft-decision. Accessed October 2026.

OpenAI. 2026. Decisions API guide. https://developers.openai.com/api/docs/guides/decisions. Accessed October 2026.

Chaoxu Pang, Yixuan Cao, Qiang Ding, and Ping Luo. 2023. Guideline learning for in-context information extraction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15372–15389, Singapore. Association for Computational Linguistics.

Nicholas Pangakis, Samuel Wolken, and Neil Fasching. 2023. Automated annotation with generative AI requires validation. *arXiv preprint arXiv:2306.00176*.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Carlos Ramisch, Silvio Ricardo Cordeiro, Agata Savary, Veronika Vincze, Verginica Barbu Mititelu, Archna Bhatia, and 1 others. 2018. Edition 1.1 of the PARSEME shared task on automatic identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Linguistic Annotation, Multiword Expressions and Constructions (LAW-MWE-CxG-2018)*, pages 222–240, Santa Fe, New Mexico, USA. Association for Computational Linguistics.

Carlos Ramisch, Agata Savary, Bruno Guillaume, Jakub Waszczuk, Marie Candito, Ashwini Vaidya, Verginica Barbu Mititelu, Archna Bhatia, Uxoa Iñurrieta, Voula Giouli, Tunga Güngör, Menghan Jiang, Timm Lichte, Chaya Liebeskind, Johanna Monti, Renata Ramisch, Sara Stymne, Abigail Walsh, and Hongzhi Xu. 2020. Edition 1.2 of the PARSEME shared task on semi-supervised identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Multiword Expressions and Electronic Lexicons*, pages 107–118. Association for Computational Linguistics.

Lance Ramshaw and Mitch Marcus. 1995. Text chunking using transformation-based learning. In *Third Workshop on Very Large Corpora*.

Omid Rohanian, Shiva Taslimipoor, Samaneh Kouchaki, Le An Ha, and Ruslan Mitkov. 2019. Bridging the gap: Attending to discontinuity in identification of multiword expressions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2692–2698, Minneapolis, Minnesota. Association for Computational Linguistics.

Oscar Sainz, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *The Twelfth International Conference on Learning Representations*.

Haibo Sun and Nianwen Xue. 2024. Anchor and broadcast: An efficient concept alignment approach for evaluation of semantic graphs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 1052–1062, Torino, Italia. ELRA and ICCL.

Haibo Sun, Nianwen Xue, Jin Zhao, Liulu Yue, Yao Sun, Keer Xu, and Jiawei Wu. 2024. Chinese UMR annotation: Can LLMs help? In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*, pages 131–139.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 930–957, Miami, Florida, USA. Association for Computational Linguistics.

Petter Törnberg. 2023. ChatGPT-4 outperforms experts and crowd workers in annotating political Twitter messages with zero-shot learning. *arXiv preprint arXiv:2304.06588*.

TypeSafe. 2026. Jev API reference. https://docs.typesafe.ai/api. Accessed October 2026.

Jens E. L. Van Gysel, Meagan Vigus, Jayeol Chun, Kenneth Lai, Sarah Moeller, Jiarui Yao, Tim O'Gorman, Andrew Cowell, William Croft, Chu-Ren Huang, Jan Hajič, James H. Martin, Stephan Oepen, Martha Palmer, James Pustejovsky, Rosa Vallejos, and Nianwen Xue. 2021. Designing a uniform meaning representation for natural language processing. *KI - Künstliche Intelligenz*, 35(0):343–360.

Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4195–4205, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Xingyao Wang, Sha Li, and Heng Ji. 2023. Code4Struct: Code generation for few-shot event structure prediction. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3640–3663, Toronto, Canada. Association for Computational Linguistics.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling*.

Daniel Zeman and Jan Štěpánek. 2026. UMR parsing shared task at DMR 2026. `https://ufal.mff.cuni.cz/umr-parsing`. Data, evaluation script and results; accessed October 2026.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2026a. Darwin Gödel machine: Open-ended evolution of self-improving agents. In *International Conference on Learning Representations*. ArXiv:2505.22954.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19632–19642.

Caleb Ziems, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. Can large language models transform computational social science? *Computational Linguistics*, 50(1):237–291.

A The seed procedures

All seeds were written from the guidelines before any training sentence was read, and contain no example: no annotated sentence, no example word in an option description, and every fragment skill's `reference/example.md` is empty. They are in `experiment/orchestration-v2/seeds/`.

UMR. Both modes share the registry: the sentence wave (`sent_type`, `discourse`, `predicate`, `np_phrase`, `special_entity`, and the rule nodes `coverage` and `antecedent`), the rule wave `reentrancy`, and the `document` wave. Each decision skill restates one part of the guidelines: `sent_type` the criteria for events (Part 3-1-1: processes in any packaging, states and entities only when predicated) and the kinds of pieces; `discourse` the lattice of discourse relations (3-1-6); `predicate` light verbs (3-1-3-5), the abstract predicates of non-verbal clauses and their roles (Tables 4 and 8), word senses (3-1-4), participant and non-participant roles (3-2), the aspect decision procedure (3-3-1: event nominals, then habitual, state, activity, endeavor, performance), polarity (3-3-3) and modal strength (4-3-1); `np_phrase` inverse roles for relative clauses, participant nominalizations and relational nouns (3-2-1-3), modifiers (3-2-2-2), quantity and unit (2-2-5) and person and number (3-3-5); `special_entity` the named-entity types of Table 5, dates and quantities; `document` the temporal superstructure and relations (4-2), the modal dependency (4-3) and coreference (4-1). In fragment mode each is a `skill.md` of 2.5–6.5k characters (35k in all) with the output schema; in closed mode each is a Python file that builds the questions and rules, with the model-facing text in a `prompts.yaml`. Function-word, pronoun and numeral tables are closed-class grammar, not annotation decisions.

PARSEME. A rule node proposes candidates from the input's dependency tree: in English a verb with its particle (`compound:prt`), with the head noun of its object or oblique, or with the adposition of its oblique; in Chinese a multi-character verb token, a verb with the verb or adjective right after it that depends on it, a verb with its object, and four-character tokens. In closed mode, each candidate is asked the yes/no tests of its kind from the guidelines' decision trees (LVC.0–LVC.5, VID.1–VID.5, VPC.1–VPC.3, IAV, and the Chinese MVC tree), and rules walk the tree on the answers with one threshold. In fragment mode the LLM reads a `decide.md` that restates the categories and tests, sees the candidates, and returns the expressions of the sentence, which may include expressions the rules did not propose.

B Reflection

The reflector's instructions (abridged): the procedure follows a written guideline, code cuts, enumerates and assembles, decision nodes judge; list every problem you can find, however small, with its cause in the procedure; propose edits (`replace`,

Algorithm 1 Reflection on five gold sentences per round.

Require: seed V_0, training batches $B_1, \ldots, B_{10}$, mode $\in$ {closed, fragment}
for $r = 1, \ldots, 10$ **do**
  $V \leftarrow V_{r-1}$; parse B_r with V; judge; attribute each difference to a step
  $P, E \leftarrow$ REFLECTOR(gold, prediction, differences, decisions, files of V, history)
  for each edit $e \in E$ **do**
    if e applies, the procedure loads, and e copies no test text **then**
      if mode = closed **then**
        $\Delta_b, \Delta_g \leftarrow$ change of F on B_r and on $B_1..B_{r-1}$ with e
        keep e iff $(\Delta_b > 0 \wedge \Delta_g \geq -0.5) \vee (\Delta_g > 0 \wedge \Delta_b \geq 0)$
      else keep e
      end if
    end if
  end for
  closed: send rejected edits back once with their measured effect; try the rewrites
  for each file f longer than $1.5\times$ its seed **do**
    rewrite f shorter (reflector); closed: keep it iff no set falls by more than 0.5
  end for
  $V_r \leftarrow V$ (frozen; tested on the test set)
end for

`insert_after`, `delete`, each an exact anchor in one file) that fix them on these sentences and on unseen text; prefer general rules and better option descriptions to special cases; keep every file valid; when you add a rule, delete or merge what it makes redundant. The closed-mode variant lists the useful kinds of edits (the right answer among the options with a rule branch; sharper option descriptions; questions added, split or removed; thresholds; conventions written into rules; code that cuts or assembles wrongly), the fragment-mode variant its own (conventions, misleading instructions removed, explicit decision procedures, output format). The full prompts are in `code/reflect.py`.

C Cascading uncertain decisions to a small LLM

Method. Every closed answer carries a confidence $c = \max_o p(o)$. In the cascade, the questions a decision model answers with $c < \tau$ are asked again, with the same state and options, of a small LLM (Claude Haiku 5.5) that must pick one of the options; its pick replaces the decision model's answer, and nothing else changes. τ is chosen on the 50 training sentences among 0.6, 0.7 and 0.8, and the cascade is used only if some τ improves the training score. We ran it for Jev only, on its seed V_0 and its final V_{10}; reflection itself ran without the cascade.

	UMR (juːmætʃ F1)		PARSEME (MWE F1)	
	Chinese	English	Chinese	English
Training sentences, Jev V_{10}				
no cascade	37.5	38.5	88.2	76.7
$\tau = 0.6$	37.0	38.0	82.2	64.6
$\tau = 0.7$	37.2	38.2	77.3	73.5
$\tau = 0.8$	37.8	38.8	78.5	69.6
selected τ	0.8	0.8	none	none
Test sentences				
Jev V_0	33.9	29.8	37.8	22.5
Jev V_0 + cascade	35.5	31.4	39.8	25.8
Jev V_{10}	37.6	31.8	63.4	53.7
Jev V_{10} + cascade	38.1	32.6	53.4	44.1

Table 6: The cascade from Jev to Haiku 5.5. Training: Jev's V_{10} on the 50 training sentences without the cascade and at three values of τ. Test: Jev's V_0 and V_{10} without and with the cascade, at the selected τ (UMR: 0.8) or, where no τ helped on the training sentences (PARSEME), at $\tau = 0.7$.

Results. Table 6 gives the results. On PARSEME the cascade hurts. On the training sentences no τ improved Jev's V_{10} (English 76.7 without the cascade, 64.6–73.5 with it; Chinese 88.2 against 77.3–82.2), and at $\tau = 0.7$ the test score falls too (English 53.7 $\rightarrow$ 44.1, Chinese 63.4 $\rightarrow$ 53.4), while on the seeds it helps a little (37.8 $\rightarrow$ 39.8, 22.5 $\rightarrow$ 25.8). Once reflection has tuned PARSEME's rules to Jev's answers, replacing the hard ones with another model's breaks that fit. On UMR, whose closed questions are more open (which concept, which role), the training sentences select $\tau = 0.8$ in both languages (Chinese 37.5 $\rightarrow$ 37.8, English 38.5 $\rightarrow$ 38.8), and the cascade adds a little on the test set too: Chinese 37.6 $\rightarrow$ 38.1 and English 31.8 $\rightarrow$ 32.6, for \$0.18–0.67 per hundred sentences, still a fraction of Haiku's own cost as a fragment decider. On the seeds it adds 1.6 points in both languages (33.9 $\rightarrow$ 35.5, 29.8 $\rightarrow$ 31.4). The cascade was not tried with the other decision models, with other escalation targets, or during reflection, so these results describe one configuration only.

D Data details

UMR. Splits are frozen by `code/splits.py` (seed 20261012) in `experiment/orchestration-v2/data/`. Chinese training: one batch of five consecutive sentences from each of ten training documents among `chinese_umr-0001`–`0018`; Chinese test: 20 chunks of five consecutive sentences from the 13 blind-test documents with gold (`0021`–`0024`,

0027–0035), at least one chunk per document. English training: five, three and two batches from english_umr-0004, 0578 and 0579; English test: all 95 sentences of english_umr-0005–0008 in 20 chunks. Input tokens are the gold Words lines, as in the shared task. Gold is rewritten in the strict format the official scorer reads (one graph per sentence, alignments restricted to graph variables, document-level relations restricted to the chunk). Scoring: compare_umr.pl -tsv, with and without -no-document-level.

PARSEME. Chinese: PARSEME 1.2 ZH; English: PARSEME 1.1 EN. Training batches: three sentences with at least one VMWE and two without, ten batches. Test: 50 sentences with and 50 without a VMWE, drawn from the official test file. Unseen F uses a training file reduced to the sentences with a VMWE, which gives the same seen/unseen split as the full file and runs in seconds. The shared-task systems' released predictions are re-scored on the same 100 sentences with the official evaluate.py.

Guidelines. The UMR guidelines (umr4nlp/umr-guidelines, guidelines.md, 366k characters) and the PARSEME 1.2 guidelines (335k characters) are given to the baseline in full and were the only source of the seeds.

E All test scores

F Fine-grained judge results

G Costs

H Earlier experiments: fragment decisions by Claude Sonnet 5.5

Before the experiments of the main text, we ran the fragment-mode UMR procedure with a stronger parser, Claude Sonnet 5.5, on Chinese UMR, with an earlier hand-written seed that contained worked examples, an earlier test protocol (whole documents; AnCast++ instead of the shared task's metric) and unmeasured reflection by Claude Opus 5.5 on one whole training document per round. These numbers are not comparable with the main text (different seed, test documents, metric and gold exposure) and are reported for completeness.

One training document raised the first test set from 45.3 to 50.8 F and a second to 52.6, with temporal relations improving most; third and fourth documents left the test scores flat. Unmeasured reflection accepted 440 of 441 edits and the skills grew from 41k to 156k characters in ten rounds, which motivated the length compression of the main text.

Task	Decisions	V_0	V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8	V_9	V_{10}
UMR zh	Haiku 5.5	20.9	27.8	30.5	31.1	31.9	32.4	31.1	33.5	33.6	33.5	33.2
	Jev 1.13	33.9	35.6	35.9	36.8	36.7	36.5	36.3	37.3	37.2	37.2	37.6
	OpenAI Decisions	33.2	34.3	34.3	34.6	34.8	34.9	35.3	35.4	35.6	35.5	35.6
	Microsoft-Decision-1	35.0	34.8	35.1	35.1	34.8	35.4	35.2	35.0	34.9	34.9	35.8
UMR en	Haiku 5.5	28.8	29.4	29.5	30.5	31.4	31.1	31.3	30.8	30.7	30.6	32.6
	Jev 1.13	29.8	30.1	31.0	30.8	31.0	31.3	31.7	31.4	31.0	31.6	31.8
	OpenAI Decisions	27.6	27.3	27.4	27.4	27.7	28.1	27.7	27.8	27.3	28.4	28.4
	Microsoft-Decision-1	29.0	29.1	30.4	30.6	30.1	31.1	31.3	30.8	30.2	31.0	31.0
PARSEME zh	GPT-6-luna	62.5	62.5	67.8	67.2	72.9	73.4	74.1	70.6	69.2	70.6	72.5
	Haiku 5.5	59.5	57.1	62.9	62.9	53.1	51.6	47.2	54.7	47.2	51.6	53.2
	Jev 1.13	37.8	36.0	36.0	52.2	56.1	56.1	56.1	56.1	56.1	56.1	63.4
	OpenAI Decisions	35.1	32.0	32.0	44.0	54.9	54.9	54.9	54.9	54.9	54.9	54.5
	Microsoft-Decision-1	36.7	36.7	34.3	39.4	40.0	40.0	46.0	46.0	46.0	46.0	46.0
PARSEME en	GPT-6-luna	62.3	61.2	57.6	60.7	55.5	60.8	58.1	59.5	55.3	59.8	58.4
	Haiku 5.5	57.6	57.4	55.9	59.5	58.8	61.3	61.1	60.0	56.6	57.9	58.2
	Jev 1.13	22.5	36.5	38.4	39.7	40.0	45.9	53.2	53.7	53.7	53.7	53.7
	OpenAI Decisions	42.0	42.0	48.6	48.2	48.2	47.4	48.6	54.0	54.0	56.6	56.6
	Microsoft-Decision-1	25.9	47.2	47.2	47.2	47.2	47.2	46.7	46.7	46.7	44.0	43.5

Table 7: Test F1 of every frozen version (UMR: juːmætʃ; PARSEME: MWE-based). Blank: not tested (budget).

	Chinese				English			
	Haiku (fragm.)		Jev (closed)		Haiku (fragm.)		Jev (closed)	
Check	V_0	V_{10}	V_0	V_{10}	V_0	V_{10}	V_0	V_{10}
event concept	30.0	77.3	73.5	77.9	66.7	72.5	49.4	55.6
entity concept	36.3	73.8	64.4	69.7	69.5	76.3	68.1	72.5
granularity	82.5	85.8	92.2	92.9	84.9	88.1	90.3	90.7
event sense	58.6	84.7	84.4	86.1	63.2	71.9	56.8	60.9
aspect	60.5	65.7	51.7	63.1	52.0	61.3	55.2	57.5
polarity	52.6	61.1	56.4	49.0	36.8	36.8	32.4	37.8
refer (person/number)	20.5	23.2	42.6	49.0	35.3	49.9	35.8	37.7
core args of events	22.2	45.6	37.8	35.7	27.5	29.5	23.0	24.9
peripheral args of events	12.3	25.3	15.4	20.9	9.0	17.6	16.8	14.0
core role labels	60.2	75.1	74.8	68.6	65.4	60.9	69.7	71.8
name tokens	50.3	52.0	46.3	52.2	61.6	67.6	45.4	46.9
reentrancy	9.7	24.4	18.3	20.4	11.4	12.6	9.8	10.5
temporal	4.5	11.1	16.1	18.5	6.7	14.5	13.7	16.8
modal	48.7	54.6	55.2	56.5	28.6	31.7	28.1	27.2
coreference	3.0	16.4	4.3	12.7	11.9	22.5	9.3	11.3
juːmætʃ F1 (official)	20.9	33.2	33.9	37.6	28.8	32.6	29.8	31.8

Table 8: The numbers of Figure 5: the fine-grained UMR judge (F1 per check, %).

Lang.	System	correct	wrong cat.	partial	missed, no cand.	missed, rejected	spurious	F
Chinese	GPT-6.1-sol	25	3	4	27	0	21	50.0
	GPT-6-luna V_0	23	12	1	6	17	17	62.5
	GPT-6-luna V_{10}	33	4	1	6	15	5	72.5
	Haiku 5.5 V_0	26	7	4	3	19	16	59.5
	Haiku 5.5 V_{10}	22	3	3	5	26	8	53.2
	Jev 1.13 V_0	7	28	12	8	4	80	37.8
	Jev 1.13 V_{10}	22	10	1	11	15	9	63.4
	OpenAI Decisions V_0	15	12	10	8	14	59	35.1
	OpenAI Decisions V_{10}	18	6	0	11	24	5	54.5
	Microsoft-Decision-1 V_0	9	24	11	8	7	78	36.7
	Microsoft-Decision-1 V_{10}	16	13	5	8	17	34	46.0
English	GPT-6.1-sol	46	5	4	2	0	22	75.6
	GPT-6-luna V_0	31	7	8	6	5	21	62.3
	GPT-6-luna V_{10}	27	6	8	7	9	17	58.4
	Haiku 5.5 V_0	30	4	8	7	8	19	57.6
	Haiku 5.5 V_{10}	25	7	9	6	10	14	58.2
	Jev 1.13 V_0	19	6	20	10	2	112	22.5
	Jev 1.13 V_{10}	23	6	4	16	8	19	53.7
	OpenAI Decisions V_0	15	10	7	17	8	27	42.0
	OpenAI Decisions V_{10}	19	11	2	14	11	18	56.6
	Microsoft-Decision-1 V_0	17	13	15	11	1	123	25.9
	Microsoft-Decision-1 V_{10}	10	10	5	18	14	11	43.5

Table 9: The numbers of Figure 6: PARSEME error kinds on the test set (counts) and MWE-based F.

	UMR		PARSEME	
Decisions	zh	en	zh	en
GPT-6.1-sol + guideline	7.09	5.09	4.05	4.02
Haiku 5.5 (fragment)	1.88	0.83	0.03	0.02
GPT-6-luna (fragment)	–	–	0.01	0.01
Jev 1.13 (closed)	0.28	0.10	0.01	0.01
Microsoft-Decision-1 (closed)	0.19	0.07	0.01	0.01

Table 10: API cost (USD) of annotating the test set (95–100 sentences) with the seed procedure, uncached, as billed through OpenRouter. Reflection used Claude Sonnet 5.5 through a subscription; its list-price equivalent is $0.4–3.2 per ten-round chain.

System	Refl. docs (sent.)	T1: 0004–05 F/ Comp.	T2: 0005–07 F/ Comp.
Single call, Sonnet 5.5	0	32.7 / 27.7	29.4 / 24.9
R_0 (seed), Sonnet 5.5	0	45.3 / 44.0	–
R_{13}	1 (51)	50.8 / 49.4	–
R_{23}	2 (111)	52.6 / 51.2	–
R_{32}	3 (142)	52.6 / 51.1	55.2 / 52.1
R_{42}	4 (201)	n/a	55.6 / 52.2

Table 11: Earlier Chinese UMR runs with Sonnet 5.5 fragment decisions (AnCast++ sentence-graph F and comprehensive score, macro-averaged over documents). R_{xy}: the version after round y on training document x. "n/a": 0004 was a training document by then.

Doc	seed	iter 1	iter 2	iter 3	after
0001	40.3	40.3	59.6	66.9	70.9
0002	50.6	58.9	66.3	70.4	73.4
0003	49.7	57.5	69.5	–	–
0004	39.6	49.7	57.9	–	–

Table 12: The same runs: sentence F on each training document over the rounds of reflection on it. Training documents gain far more than test documents, as in the main text.